



SUJET OFFICIEL

Thème : Evolving Codes – Programs that Auto-Adapt in Realtime

Date : 15 Décembre 2025

Organisateur : Dr Stella Bitchebe | Igor Mogou

1 Contexte & Objectif

1.1 Le Problème

Dans la réalité industrielle, quand on développe une application, on essaie de prévoir tous les cas possibles. Ensuite, on compile le code et on le déploie en production. À cet instant, le binaire résultat de la compilation devient un bloc statique, particulièrement encore dans les applications dites monolithiques.

Si l'on veut ajouter une fonctionnalité, corriger un bug critique ou optimiser une fonction, le cycle classique est lourd :

1. Modifier le code source.
2. Recompiler toute l'application (coûteux en temps pour de très gros logiciels).
3. **Arrêter l'ancien service** et lancer le nouveau (Interruption de service).

1.2 L'Exemple Réel

Prenons l'exemple d'une plateforme de Mobile Money gérant un volume massif de transactions. Ce type de logiciel est souvent un monolithe complexe de plusieurs millions de lignes de code.

En raison de cette taille massive, la compilation intégrale de l'application peut prendre plusieurs heures. Dans un contexte de production critique, effectuer une mise à jour complète pour une modification mineure est inenvisageable. Supposons qu'une nouvelle réglementation fiscale impose l'application immédiate d'une taxe de 0.5% sur les transactions supérieures à 10 000 FCFA. Avec l'approche classique, il faudrait recompiler tout le système et planifier une interruption de service, ce qui engendrerait des pertes financières considérables et des échecs de transactions.

L'approche industrielle moderne consiste donc à ne compiler que la nouvelle logique (la fonction de taxe) et à venir l'injecter dynamiquement dans le processus déjà en cours d'exécution, garantissant ainsi la continuité du service.

1.3 L'Objectif

Pour répondre à ce besoin, la solution est de concevoir une architecture logicielle, capable de s'auto-modifier. C'est ce que permet l'**Instrumentation Binaire Dynamique (DBI)**.

Au lieu de laisser le processeur exécuter le binaire directement, on place un **Moteur d'Instrumentation** entre le système d'exploitation et l'application. Ce moteur agit comme un superviseur qui a le pouvoir de :

- **Observer** chaque instruction avant qu'elle ne soit exécutée (Monitoring).
- **Modifier** le code en mémoire à la volée, sans toucher au fichier sur le disque.
- **Rediriger** le flux d'exécution d'une vieille fonction vers une nouvelle version plus performante ou mise à jour.

L'objectif de ce Hackathon est de construire ce moteur. Vous allez développer l'intelligence capable de détecter le besoin de changement et d'opérer l'ajout de code en temps réel, sans jamais arrêter le programme cible.

2 Objectif Détaillé

L'objectif de ce challenge est de concevoir et d'implémenter un **Moteur d'Instrumentation Binaire Adaptatif**.

Contrairement aux outils de profilage classiques qui appliquent une instrumentation uniforme (et lente) sur tout le programme, votre moteur devra faire preuve d'intelligence. Il devra être capable d'analyser dynamiquement le comportement d'un programme à l'exécution, d'identifier les zones de code critiques, et d'adapter en conséquence son niveau d'instrumentation ou la logique du programme.

2.1 Le Cycle d'Adaptation Attendu

Votre moteur doit implémenter une boucle de contrôle composée de deux phases distinctes :

Phase 1 : Analyse et Profilage (Instrumentation Légère)

Lors de la phase initiale, le moteur doit se faire discret : vous devez limiter au maximum l'impact sur les performances du programme cible (*low overhead*).

Le moteur doit collecter des métriques clés pour répondre à la question : *"Quelles sont les parties du code les plus sollicitées et que font-elles ?"*

Les métriques attendues sont :

- **Fréquence d'appel des fonctions** : Compter le nombre de fois que chaque fonction est exécutée. C'est l'indicateur principal pour repérer les zones chaudes (Hotspots).
- **Activité Mémoire (Lectures/Écritures)** : Mesurer le volume d'opérations d'échanges avec la mémoire. Une fonction qui passe son temps à lire ou écrire des données est un candidat critique pour l'optimisation.

Pour aller plus loin [pas requis, mais serait un plus] :

- **Fréquence d'utilisation des variables** : Un plus serait de descendre à une granularité plus fine, en identifiant non seulement la fonction, mais aussi combien de fois une variable spécifique (locale) est accédée.
- **Densité des Branches** : Évaluer la complexité logique en surveillant les sauts conditionnels (boucles, `if/else`).

Phase 2 : Décision et Réécriture (Adaptation)

Une fois qu'une zone de code (une fonction) dépasse un certain seuil d'activité (défini comme Hotspot), le moteur doit changer de stratégie en temps réel.

Le moteur doit être capable de :

1. **Interrompre** l'exécution de la version originale de la fonction (considérée alors comme obsolète).
2. **Rediriger dynamiquement** le flux d'exécution vers une version alternative mise à jour.

3 Outils et Contraintes

- **Architecture cible** : Linux x86_64
- **Framework d'instrumentation** : Le choix de la technologie sous-jacente est libre. Vous pouvez utiliser le framework de votre choix pour construire votre moteur, tant qu'il permet l'instrumentation binaire dynamique (DBI) sur l'architecture cible.
Suggestions d'outils : DynamoRIO, Intel Pin, Frida, `ptrace`, etc.

4 Protocole de Validation et Jeux de Données

La validation de votre moteur repose sur sa capacité à s'adapter à des situations variées sans que vous ayez besoin de modifier votre code.

4.1 Le Kit de Développement Fourni

Pour vous aider à construire et tester votre solution, nous mettons à votre disposition (via un lien Drive) un ensemble de **03 fichiers binaires distincts** (avec leurs sources .c).

Attention : Chacun de ces 3 fichiers embarque **simultanément l'ensemble des trois logiques** décrites dans les objectifs. Ils ne sont pas spécialisés par tâche, ils contiennent tous :

1. Une zone de test pour l'activité mémoire (Stack).
2. Une zone de test pour la complexité logique (Branches).
3. Un scénario complet d'optimisation (Fonction Lente vs Rapide).

La différence entre les fichiers ? Bien que la structure externe soit identique (mêmes noms de fonctions, mêmes points d'entrée), le **fonctionnement interne diffère** d'un fichier à l'autre.

4.2 L'Évaluation Finale : Le Binaire Mystère

Attention, piège à éviter : Votre moteur **ne doit pas être spécifique** aux 03 binaires fournis.

L'évaluation finale par le jury se fera sur un **4ème binaire inédit**, qui ne vous sera pas fourni.

- Il respectera strictement la même structure que les exemples (mêmes noms de fonctions).
- Cependant, sa complexité interne sera **plus élevée** et imprévisible (mélange de comportements, seuils de déclenchement différents).

Si votre moteur contient des constantes magiques (ex : `if (count == 1000)`) qui ne marchent que sur les fichiers d'exemples, il échouera au test final. **Votre solution doit être véritablement adaptative.**

5 Modalités de Rendu et Livrables

La soumission de votre projet se fera exclusivement via un **dépôt GitHub**.

5.1 Contenu du Dépôt

Votre dépôt doit être autonome et contenir :

- **Le Code Source** : L'intégralité des fichiers sources de votre moteur.
- **Le Fichier README.md** : Ce document est capital. Il doit impérativement contenir :
 1. **Guide de Démarrage** : Les lignes de commandes exactes (copier-coller) pour compiler le projet et le lancer sur l'un des binaires fournis.
 2. **Explication de la Solution** : Une description concise mais technique de votre approche. Comment détectez-vous les métriques ? Quelle méthode utilisez-vous pour x , y ou z ? et le remplacement de fonction ?

6 Grille d'Évaluation (100 Points)

Le jury évaluera les projets selon les critères suivants. Une attention particulière sera portée à la capacité du moteur à fonctionner sur le **binaire mystère**.

Catégorie	Critère	Points
1. Livrables & Installation	Conformité du Dépôt Code complet, README clair, procédure de build fonctionnelle.	5
	Reproductibilité Le projet compile et se lance du premier coup sur la machine du jury via les commandes du README.	10
2. Analyse (Phase 1)	Fréquence d'Appel Le moteur identifie correctement les fonctions les plus appelées.	10
	Activité Mémoire Le moteur détecte les fonctions faisant un usage intensif de la pile (Stack).	15
3. Adaptation (Phase 2)	Détection du Hotspot Le moteur déclenche l'adaptation au bon moment (seuil dynamique ou fixe atteint).	10
	Le Hot-Patching (Redirection) Le moteur réussit à rediriger l'exécution vers la version mise à jour. Le programme ne crash pas.	25
4. Généralisation	Test du "Binaire Mystère" La solution fonctionne sur le binaire inconnu (pas de constantes magiques ou de hardcoding).	25
TOTAL		100
BONUS	Métriques Avancées Suivi des variables locales ou Densité des branches.	+10
	Low Overhead Impact minimal sur la performance avant adaptation.	+5

« Fearless is having fears. Fearless is having doubts. Lots of them. To me, Fearless is living in spite of those things that scare you to death. »

– Taylor Swift

Bonne chance à toutes et à tous !